H3 SF2524

Anh Do, Tommaso Brambilla

anhd@kth.se , brambi@kth.se

1 Exercise 1

1.a

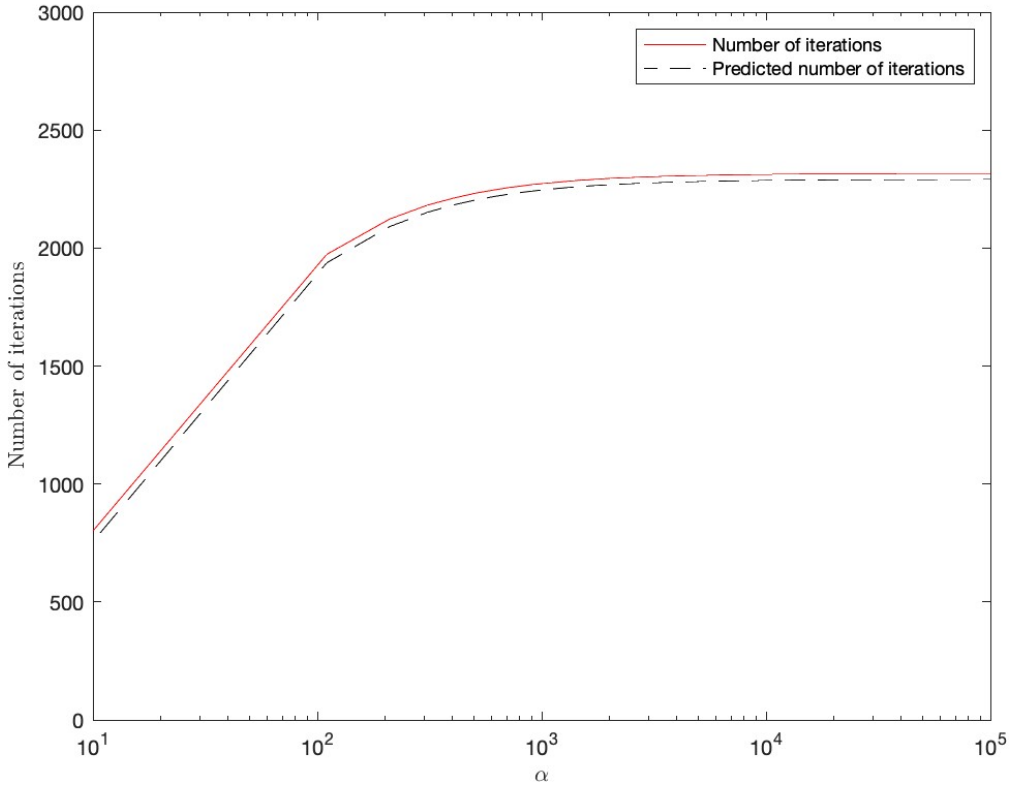

Figure 1: Number of iterations required to achieve error 10^{-10} as a function of α

1.b and 1.c

From theory, we know that the elements of the matrix A_k (k is the index of the iterations) below the diagonal will converge to zero according to:

$$|a_{ij}^{(k)}| = \mathcal{O}((\frac{|\lambda_i|}{|\lambda_j|})^k) \qquad i < j$$

where $|\lambda_1| < |\lambda_2| < ... < |\lambda_m|$ are the eigenvalues of matrix A.
So, since we defined the error at each iteration k as the largest absolute value of the matrix A_k below the diagonal, the the error will be dominated by the largest value of:

$$(\frac{|\lambda_i|}{|\lambda_j|})^k, \quad i < j, \qquad i, j = 1, 2, .., m$$

Analyzing the eigenvalues of the matrix A with respect to the value of α, we notice that the largest value of $\frac{|\lambda_i|}{|\lambda_j|}$ is obtained when $i = m-1$ and $j = m$. In particular, for large α, this ratio approaches the value 0.99, meaning that to reach an accuracy of 10^{-10} we expect a number of iterations:

$$k \approx \frac{log(10^{-10})}{log(0.99)} \approx 2290$$

This result justifies the number of iterations obtained in the graph for large values of α

2 Exercise 2

2.a

Generalized lemma:
Lemma: suppose $x \in R^m \setminus \{0\}$, $y \in R^m \setminus \{0\}$ and let $\rho = \pm 1$ and $\alpha = \rho\frac{\|x\|}{\|y\|}$. Let

$$u = \frac{x - \alpha y}{\|x - \alpha y\|} = \frac{z}{\|z\|}$$

where

$$z = x - \alpha y = \begin{bmatrix} x_1 - \rho\frac{\|x\|}{\|y\|}y_1 \\ x_2 - \rho\frac{\|x\|}{\|y\|}y_2 \\ \cdot \\ \cdot \\ \cdot \\ x_m - \rho\frac{\|x\|}{\|y\|}y_m \end{bmatrix}$$

Then, the matrix $P = I - 2uu^T$ is a Householder reflector and $Px = \alpha y$

2.b

See code file.

2.c

The difference in time is mainly given by the multiplications done in each iteration. The naive method computes two matrix-matrix products ($A_k = P_k A_{k-1} P_k^T$) at each iteration of the loop, while in algorithm 2, we only compute matrix-vector products.
In both cases, CPU-time increases with the size of the matrix, since products between larger matrices increase the number of computations.

	CPU-time Algorithm 2	**CPU-time of algorithm in (b)**
m=10	0.0003	0.0003
m=100	0.0051	0.0217
m=200	0.0380	0.0952
m=300	0.1060	0.3821
m=400	0.2162	1.1636

Table 1: Comparison of Naive and Improved Hessenberg Reduction Times

2.d

We implemented the algorithm of the shifted QR-method considering the two-phase approach.
When $\sigma = 0$, the shifted QR is equivalent to the QR-method solved with the two-phase approach.
We can see that, after just one iteration of the shifted QR, the value of $|\bar{h}_{2,1}|$ decreases as ϵ decreases. We know from the theory that when the shift is exact, so it is equal to an eigenvalue of A, then the method only takes one iteration to converge. So, as ϵ gets smaller, A matrix gets closer to an upper triangular matrix, meaning that the eigenvalues are closer to the elements on the diagonal, in particular $\sigma = a_{2,2}$ will be an eigenvalue approximation (in fact, $|\bar{h}_{2,1}| = 0$ in the table below, when $\epsilon = 0$).
In this case, it is favorable to choose $\sigma = 1$, since after one iteration of the shifted QR, the $\bar{H}$ matrix is more similar to an upper triangular matrix than it is with QR-method ($\sigma = 0$). For this reason, we can state that when the shift is equal to 1, the method will converge faster.

ϵ	$\sigma = 0$	$\sigma = a_{2,2} = 1$
0.4	0.096069868995633	0.076923076923077
10^{-1}	0.031076581576027	0.004987531172070
10^{-2}	0.003311074321396	0.000049998750031
10^{-3}	0.000333111074099	0.000000499999875
10^{-4}	0.000033331111074	0.000000005000000
10^{-5}	0.000003333311111	0.000000000050000
10^{-6}	0.000000333333111	0.000000000000500
10^{-7}	0.000000033333331	0.000000000000005
10^{-8}	0.000000003333333	0.000000000000000
10^{-9}	0.000000000333333	0.000000000000000
10^{-10}	0.000000000033333	0.000000000000000
0	0	0

Table 2: Values of $\left|\bar{h}_{2,1}\right|$ for different ϵ and σ

3 Exercise 3

3.a

$$sin(A) = \begin{bmatrix} 0.8023 & -0.2700 & -0.6142 \\ 0.1201 & 0.4211 & -0.3961 \\ -0.1691 & -0.0980 & 0.5353 \end{bmatrix}$$

3.b

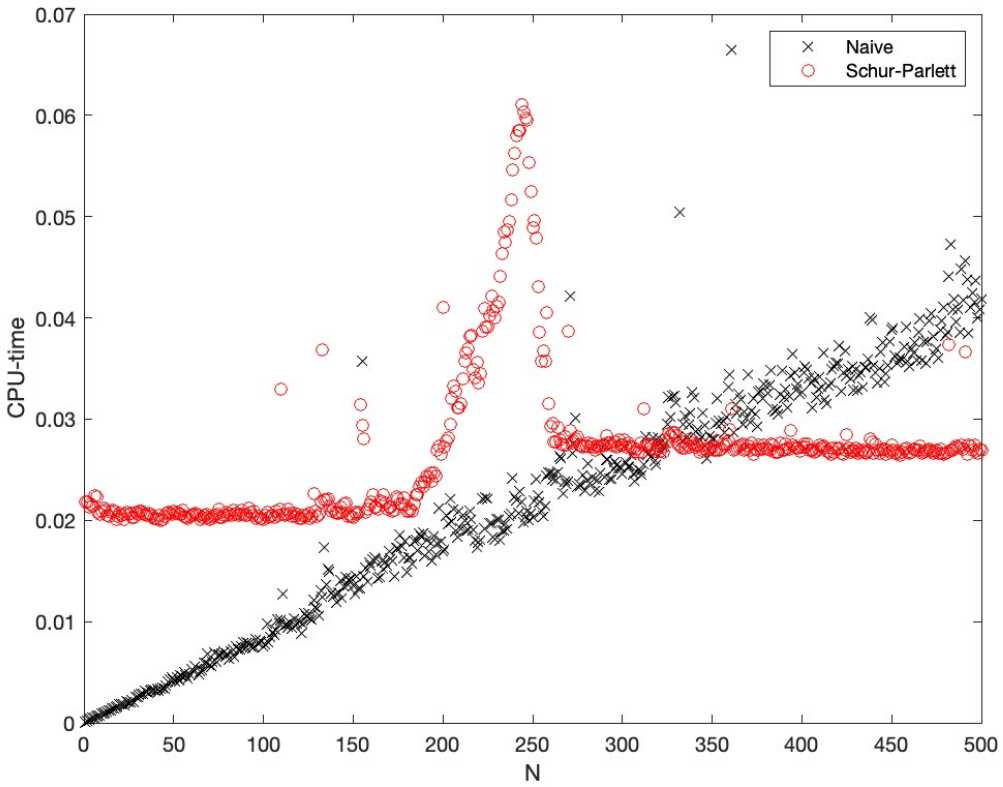

Figure 2: Naive and Schur-Parlett methods' CPU-time to compute A^N, as a function of N

3.c

The theoretical computational complexities for the Naive method and Schur-Parlett method, as functions of the matrix size n and the exponent N, are respectively of magnitude $\mathcal{O}(n^3N)$ and $\mathcal{O}(n^3)$.
These quantities can be computed looking how the two algorithms work:
Naive method:
each iteration requires a matrix-matrix product (A is multiplied at each iteration and the number of operations of the matrix-matrix product is n^3, since we need to compute n scalar multiplications for each element of the product matrix, whose size is nxn) updating the product (N-1) times.
Schur-Parlett:
complexity is mainly driven by the Schur-decomposition ($\mathcal{O}(n^3)$), since the operations involving the triangular matrices are less expensive. Moreover, the number of operations in the algorithm is not a function of N.

We can verify from the graph that the complexity, measured by CPU-time, is a linear function of N for the naive algorithm, while the CPU-time of the Schur-Parlett method is almost a constant with respect to N. Moreover, the naive method shows some stability issues when N gets larger.

4 Exercise 4

4.a

Proof that if p is a polynomial which interpolates g in the eigenvalues of the matrix

$$A = \begin{bmatrix} \pi & 1 \\ 0 & \pi + \epsilon \end{bmatrix}$$

then, $p(A) = g(A)$.
Proof:
Since A is a triangular matrix, its eigenvalues are represented by the elements on the diagonal: $\pi \quad \pi + \epsilon$.
Then, since all the eigenvalues of A are different, A is diagonalizable. This means that, the Jordan form will be equivalent to the diagonal matrix

$$J = \begin{bmatrix} \pi & 0 \\ 0 & \pi + \epsilon \end{bmatrix}$$

and we can write $A = PJP^{-1}$ where P is the matrix whose columns are the eigenvectors of A:

$$P = \begin{bmatrix} 1 & 1 \\ 0 & \epsilon \end{bmatrix}$$

Now, because of the properties of the Jordan form, we know that a matrix function commutes with similarity transformations in a way that we can write:

$$p(PJP^{-1}) = Pp(J)P^{-1}$$

moreover, since J is a diagonal matrix:

$$p(J) = \begin{bmatrix} p(\pi) & 0 \\ 0 & p(\pi + \epsilon) \end{bmatrix}$$

But from the hypotesis we know that $p(\pi) = g(\pi)$ and $p(\pi + \epsilon) = g(\pi + \epsilon)$, implying that

$$p(J) = g(J)$$

For these reasons we can show that $p(A) = g(A)$:

$$p(A) = Pp(J)P^{-1} = Pg(J)P^{-1} = g(PJP^{-1}) = g(A)$$

Now we want to find the exact expressions for α and β when $p(z) = \alpha + \beta z$ in terms of g, for the matrix A. We can do it by solving the following system:

$$p(\pi) = \alpha + \beta\pi = g(\pi)$$

$$p(\pi + \epsilon) = \alpha + \beta(\pi + \epsilon) = g(\pi + \epsilon)$$

The values obtained are:

$$\beta = \frac{g(\pi + \epsilon) - g(\pi)}{\epsilon} \qquad \alpha = g(\pi) - \beta\pi = \frac{\pi + \epsilon}{\epsilon} g(\pi) - \frac{\pi}{\epsilon} g(\pi + \epsilon)$$

4.b

Now we show a formula for the matrix-function $exp(A)$ using the results from the previous point:

$$exp(A) = exp(PJP^{-1}) = Pexp(J)P^{-1}$$

where

$$exp(J) = \begin{bmatrix} e^{\pi} & 0 \\ 0 & e^{\pi+\epsilon} \end{bmatrix}$$

$$P^{-1} = \begin{bmatrix} 1 & \frac{-1}{\epsilon} \\ 0 & \frac{1}{\epsilon} \end{bmatrix}$$

Computing the matrix-matrix products $Pexp(J)$ and $(Pexp(J))P^{-1}$ we get the final result:

$$exp(A) = \begin{bmatrix} e^{\pi} & \frac{e^{\pi+\epsilon}-e^{\pi}}{\epsilon} \\ 0 & e^{\pi+\epsilon} \end{bmatrix}$$

4.c

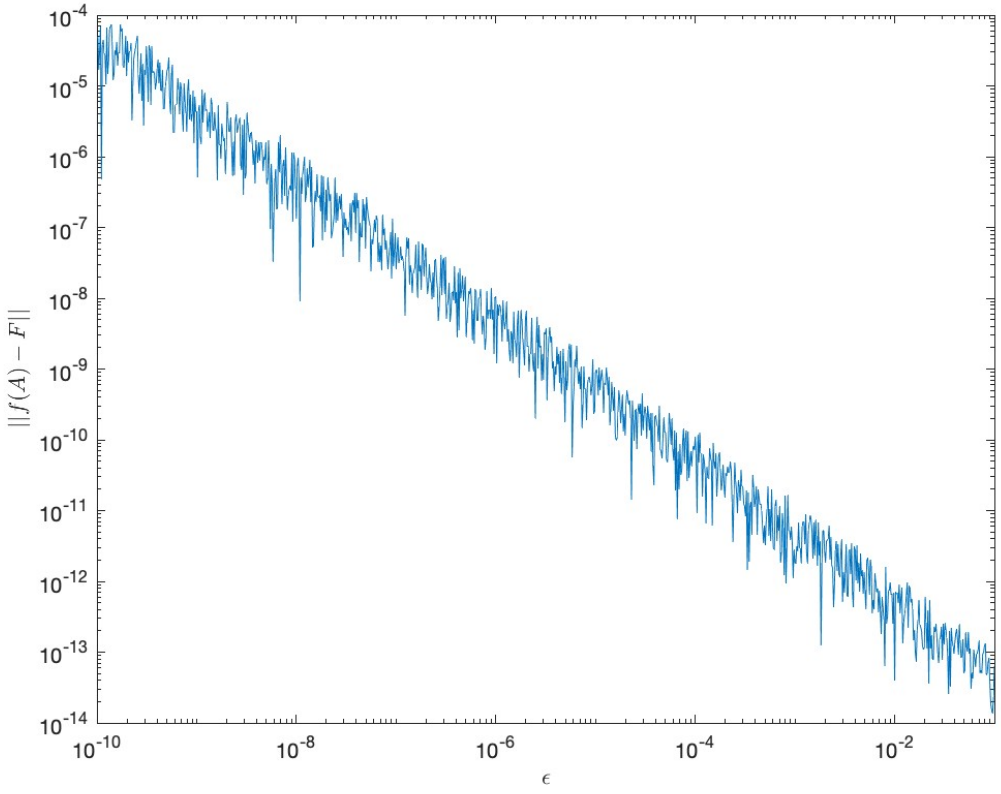

Figure 3: Norm of the difference between computing $exp(A)$ with the Jordan definition (F) and the exact result from point 4.b ($f(A)$) with respect to ϵ

From the figure we can see that $\|f(A) - F\|$ is larger for smaller values of ϵ. This is due to the fact that when ϵ approaches 0, the eigenvalues of matrix A are almost the same (π), leading to a jordan block greater than 1 and thus implying non-diagonalizable. This also leads to a badly conditioned eigenbase that is near singular for our F method. In combination with our $f(A)$ method which approaches infinity as we get division by zero as $\epsilon \to 0$.

5 Exercise 5

5.a

Each element of the first table has the value $(m-1)+j$, because the scaling part of the algorithm requires $m-1$ matrix-matrix products (since it consists of a Taylor expansion of m terms) while the squaring part takes

j matrix-matrix products (since squaring a matrix j times means to multiply the matrix by itself j times). About the second table: it is reasonable to see that the values decrease as both j and m get larger, meaning that the norm of the difference is getting smaller. This fact is justified from the theory, since we know that Taylor expansion converges faster when the argument of the function is smaller (when j is bigger) and the value of P_{ij} is more similar to the actual exponential function when we take in consideration more terms of the Taylor expansion ($m+1$ is the number of terms).

	$j=0$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
$m=1$	0	1	2	3	4	5	6	7
$m=2$	1	2	3	4	5	6	7	8
$m=3$	2	3	4	5	6	7	8	9
$m=4$	3	4	5	6	7	8	9	10
$m=5$	4	5	6	7	8	9	10	11
$m=6$	5	6	7	8	9	10	11	12
$m=7$	6	7	8	9	10	11	12	13

Table 3: Total number of matrix-matrix products

	$j=0$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
$m=1$	0.4424	-0.1193	-1.1479	-1.4552	-1.7618	-2.0659	-2.3686	-2.6704
$m=2$	0.6262	-0.1375	-1.4032	-2.2803	-2.9631	-3.5991	-4.2171	-4.8269
$m=3$	0.6495	-1.0885	-2.2179	-3.2517	-4.2318	-5.1741	-6.0968	-7.0097
$m=4$	0.5573	-1.2054	-2.8916	-4.2727	-5.5615	-6.8077	-8.0327	-9.2473
$m=5$	0.3743	-1.8960	-3.7009	-5.3789	-6.9706	-8.5189	-10.0456	-11.5615
$m=6$	0.1165	-2.3960	-4.5710	-6.5541	-8.4483	-10.2984	-12.1265	-13.9324
$m=7$	-0.2050	-3.0399	-5.5034	-7.7894	-9.9857	-12.1374	-14.2562	-14.2027

Table 4: $\mathrm{Log}\|P_{j,m} - \exp(A)\|$

5.b

	$j=0$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
$m=1$	0.0071	0.0071	0.0078	0.0083	0.0087	0.0091	0.0093	0.0097
$m=2$	0.0075	0.0078	0.0084	0.0093	0.0101	0.0103	0.0104	0.0106
$m=3$	0.0078	0.0090	0.0088	0.0094	0.0097	0.0102	0.0105	0.0107
$m=4$	0.0083	0.0087	0.0093	0.0096	0.0101	0.0104	0.0106	0.0109
$m=5$	0.0087	0.0092	0.0096	0.0101	0.0104	0.0109	0.0111	0.0114
$m=6$	0.0091	0.0097	0.0101	0.0105	0.0107	0.0110	0.0114	0.0117
$m=7$	0.0094	0.0099	0.0103	0.0108	0.0110	0.0114	0.0116	0.0118

Table 5: CPU-time

5.c

Based on our tables, if we want an error smaller than 10^{-12}, at least 11 matrix-matrix products are needed. We can justify this statement looking at the first two tables: from the second one, we see that the only combinations of m and j leading to an error smaller than 10^{-12} ($\iff$ the value in the table is < -12) are:

- m=6,j=6
- m=6,j=7
- m=7,j=5
- m=7,j=6
- m=7,j=7

and the corresponding number of matrix-matrix products for these values of m and j are respectively: 11,11,12,12,13
We need $m = 25$ to reach an accuracy of 10^{-12} when $j = 0$.
CPU-time to achieve this result when $j = 0$ is 0.0172 seconds. The CPU-time is almost linear in m, since the number of matrix-matrix products is linear in m.

6 Exercise 6

6.a video 312

Given a linear system $Ax = b$ of dimension (nxn), in order to compute the Hessenberg reduction of the matrix A (first phase of the two-phase approach for the QR method), we want to build a square (nxn) matrix P such that, given a vector $x \in R^n$ the following equality holds:

$$Px = \alpha e_1$$

where α is a scalar and $e_1 \in R^n$ is the standard vector with the first element equal to 1 and the others equal to 0.
We can formalize this statement with the following lemma:
Lemma: Suppose $x \in R^n$ and let

- $\rho = \pm 1$
- $\alpha = \rho \|x\|$
- $z = x - \alpha e_1$
- $u = \frac{z}{\|z\|}$

then, $P = I - 2uu^T$ satisfies $Px = \alpha e_1$

6.b video 325

In the second phase of the two-phase QR method we want to compute the QR-factorization of the Hessenberg matrix obtained after phase one. Using the Givens rotators we can explicitly write down the QR-factorization in a compact form. Formally:
Theorem: Suppose A is an unreduced Hessenberg matrix. Define

$$H_1 = A \qquad H_{i+1} = G_i^T H_i \quad \forall i = 1, \ldots, n-1$$

where G_i is the Givens rotator $G(i, i+1, c_i, s_i)$ and the parameters are defined as:

$$c_i = \frac{(H_i)_{i,i}}{\sqrt{((H_i)_{i,i})^2 + ((H_i)_{i+1,i})^2}} \qquad s_i = \frac{(H_i)_{i+1,i}}{\sqrt{((H_i)_{i,i})^2 + ((H_i)_{i+1,i})^2}}$$

Then, H_n is upper triangular and $A = G_1 G_2 \cdot \cdots \cdot G_{n-1} H_n$ is a QR-factorization.

6.c video 333

In the video the convergence of the shifted QR-method with exact shift is analyzed (shift μ equal to one of the eigenvalues of the matrix A).
We can prove that after only one step, we get a matrix in blocks were the eigenvalues of A are the eigenvalues of the top-left and low-right blocks of A_1, and the low-right block consists only of μ, proving that the shifted QR-method converges in just one step.
This result shows that when choosing the shift, we get faster convergence when it is closer to an eigenvalue of A.

6.d video 344

Through NSI (Normalized Simultaneous Iteration) and USI (Unnormalized Simultaneous Iteration) algorithms we can get a generalization of the power method to matrices. USI and NSI are equivalent (have the same iterations) if we add a final "normalization" step at USI, consisting of the QR-factorization of the matrix obtained at the k-th iteration $V^{(k)} : Q^{(k)} R^{(k)} = V^{(k)}$. In particular, analyzing NSI, we can see that

$$Q^{(k)} R^{(k)} R^{(k-1)} \cdots R^{(1)} = A^{(k)} Q^{(0)}$$

and this is just the QR-factorization of $A^{(k)}Q^{(0)}$, since $Q^{(k)}$ is an orthogonal matrix and $R^{(k)}R^{(k-1)}\cdots R^{(1)}$, being a product of upper triangular matrices, is an upper triangular matrix.
Considering the USI, with the "normalization" step, the iterates of USI and NSI are the same if:
$R^{(1)},\dots R^{(k)}$ are non singular and the QR-factorization is selected uniquely.

6.e video 345

In this video we show the convergence of USI under certain assumptions. We are interested in showing that the columns of the matrix $Q^{(k)}$ approximate the eigenvectors of A (which are equal to the canonical vectors $e_1, e_2, .., e_k$ since for assumption we are considering A as a diagonal matrix with all different eigenvalues).
In particular, after some computations, we found that the i-th column of $Q^{(k)}$ is

$$q_i^{(k)} = e_i + o((\frac{\lambda_2}{\lambda_1})^k) + o((\frac{\lambda_3}{\lambda_2})^k) + \cdots + o((\frac{\lambda_{i+1}}{\lambda_i})^k)$$

where $|\lambda_1| > |\lambda_2| > \cdots > |\lambda_n|$ are the eigenvalues of A. This result gives a convergence factor for Q^k

6.f video 423

The Newton SQRTM gives a way to compute the square root of a given matrix A, whose eigenvalues are all outside of the negative part of the real axis.
The method is based on the general Newton method to find the roots of the scalar function $g(x) = x^2 - a$, in order to calculate $\sqrt{a}$ and apply the same algorithm to matrices.
In this case, the iterates found by the Newton method are

$$x_{k+1} = x_k - \frac{g(x_k)}{g'(x_k)} = \frac{x_k + ax_k^{-1}}{2}$$

leading to the matrix form of the algorithm:

$$X_0 = A$$

$$X_{k+1} = \frac{X_k + AX_k^{-1}}{2}$$

where X_k is the k-th approximation of $\sqrt{A}$. When $A = A^T$ we can show that the method applied to matrices inherit the same convergence we had for scalars, so, locally it has quadratic convergence (essentially it can be considered globally).

6.g video 414b

When applying a certain function f to an upper triangular matrix T, we have a theorem that gives a way to calculate all the unknown elements of $f(T)$.
In particular we have the following theorem:
Theorem: Suppose $T \in C^{nxn}$ is an upper triangular matrix with distinct eigenvalues. Then, for any i and $j > i$

$$f_{ij} = \frac{t_{ij}(f_{jj} - f_{ii}) + \sum_{k=i+1}^{j-1} t_{ik}f_{kj} - f_{ik}t_{kj}}{t_{jj} - t_{ii}}$$

This means that to compute f_{ij} we have to know all the elements on the left and all the elements below the position i, j. Since we know that the matrix-function f of an upper triangular matrix T is an upper triangular matrix where the diagonal elements are given by the function of the diagonal elements of T, we can apply the theorem to find the values of the elements in the first upper diagonal, then repeat the procedure for the second one and so on until completing the whole matrix.